Introducción

El presente trabajo tiene como propósito desarrollar y analizar un programa en Java denominado QuindíoExpress, que simula el funcionamiento de un centro de distribución encargado de gestionar paquetes y coordinar su proceso de despacho y entrega. El proyecto permite aplicar los conceptos fundamentales de las estructuras de datos y los algoritmos mediante un caso práctico, en el que se administran paquetes, repartidores, destinos, prioridades y estados de distribución.

Para el desarrollo del sistema se emplean diferentes estructuras de datos, entre ellas listas, mapas, conjuntos, colas FIFO y colas de prioridad. Cada una cumple una función específica dentro de la organización y el procesamiento de la información. Asimismo, el programa incorpora mecanismos de búsqueda, ordenamiento y comparación de paquetes, junto con algoritmos recursivos para calcular el peso total de los paquetes dirigidos a un municipio, contar aquellos que cumplen un nivel mínimo de prioridad y determinar el paquete de mayor peso mediante la estrategia de divide y vencerás.

El sistema también contempla operaciones relacionadas con el registro de paquetes y repartidores, la asignación de paquetes según la zona de distribución, el control de los estados de cada paquete y el registro de las entregas realizadas. Adicionalmente, se implementa una lista enlazada genérica para almacenar el historial de entregas, lo que permite estudiar el funcionamiento de una estructura dinámica y su aplicación dentro de un sistema de gestión.

Finalmente, el proyecto se valida mediante la ejecución del programa y un conjunto de pruebas unitarias que permiten comprobar el comportamiento de sus principales componentes, las condiciones de sus operaciones y la consistencia entre las estructuras de datos utilizadas. De esta manera, QuindíoExpress constituye un ejercicio práctico para comprender cómo la selección e implementación adecuada de estructuras de datos y algoritmos contribuye a organizar, procesar y administrar información de manera coherente dentro de un problema computacional.

Necesidad	Estructura elegida	Operación principal	Complejidad esperada	Alternativa descartada	Justificación
Mantener los paquetes pendientes conservando exactamente el orden en que fueron registrados.	*Queue<Paquete>* implementada con *ArrayDeque<Paquete>*	Insertar al final y retirar del frente.	\(O(1)\) amortizado para insertar y \(O(1)\) para retirar.	*PriorityQueue<Paquete>*	La cola FIFO conserva el orden de llegada de los paquetes, de modo que el primero registrado es el primero que se retira de esta estructura. Una cola de prioridad puede alterar ese orden según sus criterios de comparación. En el programa, ambas colas se mantienen sincronizadas para evitar que un paquete

					retirado de una permanezca disponible en la otra.
Consultar rápidamente un paquete mediante su código.	HashMap<String, Paquete>	Buscar un paquete por su código.	\(O(1)\) esperado para consultar un paquete por clave	ArrayList<Paquete>	El mapa permite localizar un paquete mediante su código sin recorrer secuencialmente todos los elementos. En cambio, una búsqueda secuencial en un `ArrayList` requiere O(n) en el peor caso, donde n representa la cantidad de paquetes registrados.
Mantener el conjunto de municipios destino evitando duplicados.	HashSet<String>	Comprobar la existencia de un municipio e incorporar municipios nuevos al conjunto.	\(O(1)\) esperado.	ArrayList<String>	El conjunto permite evitar municipios duplicados y comprobar su existencia de manera eficiente. Una lista podría almacenar el mismo municipio varias veces y requerir una búsqueda lineal para detectar repeticiones. Esta estructura se utiliza para controlar las claves de los municipios registrados.

Mantener los municipios ordenados alfabéticamente y sin elementos repetidos.	*TreeSet<String>*	Insertar municipios, comprobar su existencia y recorrerlos en orden.	O(log d) para insertar o consultar un municipio y O(d) para recorrer el conjunto completo, donde d representa la cantidad de municipios distintos.	*HashSet<String>*	*TreeSet* mantiene los elementos ordenados según su criterio de comparación y evita duplicados. En la implementación del programa, la comparación no distingue mayúsculas y minúsculas. Por su parte, *HashSet* no garantiza un orden específico al recorrer sus elementos.
5. Atender paquetes teniendo en cuenta primero la mayor prioridad.	*PriorityQueue<Paquete>* con un Comparator<Paquete> personalizado.	Consultar y retirar el siguiente paquete según los criterios establecidos para el despacho.	\(O(1)\) para consultar el elemento prioritario; \(O(\log n)\) para insertarlo o extraerlo.	*ArrayDeque<Paquete>*	La cola de prioridad permite seleccionar los paquetes según las reglas de despacho implementadas: mayor prioridad, menor tiempo estimado y código ascendente para desempatar. Una cola FIFO no considera estos criterios, sino el orden de llegada de los paquetes.
6. Mantener un historial de paquetes entregados respetando el orden de entrega.	Lista simplemente enlazada propia.	Agregar cada paquete entregado al final del historial.	\(O(1)\) por inserción al final si se mantiene una referencia al último nodo.	*ArrayList<Paquete>*	La lista enlazada propia permite conservar el orden de las entregas y trabajar directamente con una estructura

					dinámica implementada en el proyecto. Además, permite cumplir la restricción del enunciado de utilizar una implementación propia en lugar de sustituirla por una colección estándar de Java.
7. Asociar cada código de paquete con el objeto `Paquete` correspondiente.	*Map<String, Paquete>* implementado con HashMap<String, Paquete>.	Registrar la asociación entre el código y el objeto, y recuperar el paquete mediante su clave.	\(O(1)\) esperado para insertar, consultar y actualizar.	*ArrayList<Paquete>*	El mapa establece una relación directa entre cada código y su objeto `Paquete`. Esta asociación facilita la recuperación de paquetes y la comprobación de códigos existentes sin recorrer toda la colección. Esta misma estructura también permite realizar la búsqueda rápida descrita en el ítem 2.

A.2. Tiempo frente a memoria (Comparación entre ArrayList y HashMap)

Tabla Comparativa

Criterio	*ArrayList<Paquete>*	*HashMap<String, Paquete>*
Tiempo esperado de búsqueda por código	\(O(n)\) en el peor caso, porque es necesario recorrer los elementos hasta encontrar el código buscado.	\(O(1)\) esperado, porque el código se utiliza como clave para localizar el paquete. En el peor caso, debido a colisiones, puede ser más costoso.
Memoria adicional	Relativamente baja: almacena referencias a los objetos `Paquete` en un arreglo	Generalmente mayor: almacena las claves, los valores y estructuras internas para gestionar las asociaciones y las colisiones.

	dinámico.	
Ventaja	Es sencillo de implementar, conserva el orden de inserción y permite recorrer los paquetes secuencialmente.	Permite consultar paquetes por código de forma eficiente, especialmente cuando se manejan grandes cantidades de registros.
Elección	Adecuado cuando predominan los recorridos secuenciales, se necesita conservar el orden de registro o la cantidad de paquetes es pequeña.	Adecuado cuando se necesita recuperar paquetes rápidamente mediante su código identificador.

2. Análisis del tiempo de búsqueda

En un `ArrayList`, si se busca un paquete por su código y no se utiliza otro mecanismo de búsqueda, es necesario comparar el código buscado con el de los paquetes uno por uno. En el peor caso, será necesario revisar los \(n\) elementos, por lo que la complejidad temporal es \(O(n)\).

Por ejemplo, en un escenario hipotético con 500.000 paquetes, una búsqueda secuencial podría requerir hasta 500.000 comparaciones.

En un `HashMap`, el código del paquete se utiliza como clave para recuperar el objeto asociado. Esto permite realizar búsquedas con un tiempo esperado de \(O(1)\), sin recorrer secuencialmente todos los paquetes. Sin embargo, este coste es esperado y no constituye una garantía absoluta, ya que las colisiones pueden aumentar el tiempo de búsqueda.

3. Análisis de memoria

El `ArrayList` almacena referencias a los paquetes en un arreglo dinámico. Su estructura es relativamente sencilla y permite recorrer los elementos en el orden de inserción.

El `HashMap` almacena las asociaciones entre claves y valores, además de utilizar estructuras internas para gestionar las búsquedas y las colisiones. Por esta razón, generalmente requiere más memoria adicional que un `ArrayList` que contiene referencias a los mismos objetos `Paquete`.

Este consumo adicional puede justificarse cuando se necesita consultar frecuentemente los paquetes mediante su código identificador, ya que permite mejorar la eficiencia esperada de estas búsquedas.

4. Decisión final

Para consultar los paquetes por su código, el programa utiliza `HashMap<String, Paquete>`. Esta estructura permite recuperar los objetos mediante su clave con un tiempo esperado de búsqueda de \(O(1)\), frente a la complejidad \(O(n)\) de una búsqueda secuencial en un `ArrayList`.

Aunque `HashMap` generalmente requiere más memoria adicional, esta característica representa un costo razonable cuando se prioriza la consulta eficiente por código. La diferencia puede ser especialmente relevante si el sistema crece desde los 30 paquetes utilizados en la demostración actual hasta un escenario hipotético de 500.000 paquetes.

No obstante, ambas estructuras cumplen funciones complementarias dentro de QuindíoExpress. El `HashMap`

facilita la búsqueda de paquetes por código, mientras que el `ArrayList` conserva el orden de registro. De esta manera, el programa aprovecha las ventajas de cada estructura sin exigir que una sola resuelva necesidades diferentes.

Parte C. Análisis algorítmico y pruebas

C.1. Pila de llamadas

Para analizar la ejecución recursiva se utiliza el método *sumarPesoPorDestino()* de la clase *OperacionesRecursivas.java.* Este método suma los pesos de los paquetes que pertenecen a un municipio determinado.

La función recibe una lista de paquetes y un destino. En cada llamada analiza un paquete y avanza al siguiente índice. Cuando el índice alcanza el tamaño de la lista, se llega al caso base y se retorna `0.0`. A partir de ese momento, las llamadas anteriores se resuelven en orden inverso, sumando los pesos correspondientes.

Ejemplo de prueba de escritorio

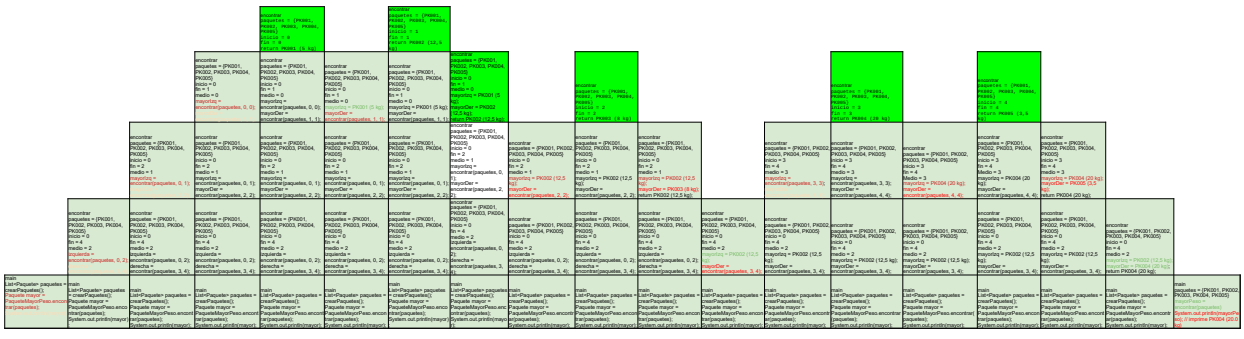
Se utiliza la siguiente lista de cinco paquetes, tomada de las pruebas de la clase *OperacionesRecursivasTest.java*, con el destino buscado «Armenia».

Índice	Código	Destino	Peso	Aporte a la suma
0	PK001	Armenia	5,0 kg	5,0 kg
1	PK002	Calarcá	3,5 kg	0 kg
2	PK003	Armenia	2,0 kg	2,0 kg
3	PK004	Montenegro	4,0 kg	0 kg
4	PK005	Armenia	1,5 kg	1,5 kg

El resultado esperado es:

Peso total para Armenia = 5,0 + 2,0 + 1,5 = 8,5 kg.

Diagrama de la pila de llamadas



La pila crece a medida que se realizan las llamadas recursivas. En este ejemplo, las llamadas avanzan desde el índice 0 hasta el índice 5, que corresponde al caso base.

```
ENTRADA
sumarPesoPorDestino(lista, "Armenia", 0)
     |
     v
sumarPesoPorDestino(lista, "Armenia", 1)
```

```
    |
    v
sumarPesoPorDestino(lista, "Armenia", 2)
    |
    v
sumarPesoPorDestino(lista, "Armenia", 3)
    |
    v
sumarPesoPorDestino(lista, "Armenia", 4)
    |
    v
sumarPesoPorDestino(lista, "Armenia", 5)
    |
    v
CASO BASE: retorna 0.0
    |
    v
Índice 4: 1.5 + 0.0 = 1.5
    |
    v
Índice 3: 0.0 + 1.5 = 1.5
    |
    v
Índice 2: 2.0 + 1.5 = 3.5
    |
    v
Índice 1: 0.0 + 3.5 = 3.5
    |
    v
Índice 0: 5.0 + 3.5 = 8.5
    |
    v
RESULTADO: 8.5 kg
```

En este proceso, cada llamada mantiene su propio índice y espera el resultado de la llamada siguiente. Una vez alcanzado el caso base, la pila se va resolviendo en orden inverso. Las llamadas que corresponden a otros municipios aportan cero, mientras que las llamadas de paquetes destinados a Armenia aportan su peso.

Este algoritmo también demuestra la importancia del caso base: sin la condición que comprueba si el índice alcanzó el tamaño de la lista, la recursión no terminaría correctamente.

C.2. Pruebas de escritorio de los algoritmos

C.2.1. Suma recursiva de pesos por destino

Se utiliza la lista de cinco paquetes presentada anteriormente y se busca el destino Armenia.

Paso	Índice	Peso del paquete	¿Destino Armenia?	Resultado acumulado al retornar
1	4	1,5 kg	Sí	1,5 kg
2	3	4,0 kg	No	1,5 kg
3	2	2,0 kg	Sí	3,5 kg
4	1	3,5 kg	No	3,5 kg
5	0	5,0 kg	Sí	8,5 kg

El resultado final es **8,5 kg**. Este procedimiento coincide con la prueba automatizada `testSumarPesoPorDestino()` de `OperacionesRecursivasTest.java`.

C.2.2. Conteo recursivo según prioridad

El método *contarPorPrioridadMinima()* cuenta los paquetes cuya prioridad es mayor o igual al umbral recibido.

En la prueba automatizada se utiliza una prioridad mínima de 5 y los siguientes datos:

Código	Prioridad	¿Prioridad mayor o igual a 5?
PK001	2	No
PK002	9	Sí
PK003	5	Sí
PK004	3	No
PK005	9	Sí

Por tanto, el resultado es **3 paquetes**. El caso base retorna cero cuando se han recorrido todos los elementos; las llamadas anteriores suman uno cuando el paquete cumple la condición y cero cuando no la cumple.

C.2.3. Búsqueda binaria por código

La clase `BusquedaPaquetes.java` realiza una copia de la lista recibida y la ordena mediante el orden natural de `Paquete`, que compara los códigos de forma ascendente. Después aplica búsqueda binaria sobre la copia ordenada.

Para ilustrar el procedimiento, se utilizan cuatro códigos del enunciado: PQ731, PQ105, PQ942 y PQ318.

Primero, se ordenan alfabéticamente:

PQ105, PQ318, PQ731, PQ942

Se busca el código PQ731.

Paso	Límite izquierdo	Límite derecho	Posición central	Código central	Decisión
1	0	3	1	PQ318	PQ731 es mayor; buscar en la mitad derecha.
2	2	3	2	PQ731	Se encuentra el paquete.

La búsqueda encuentra PQ731 en dos comparaciones. Si el código no existiera, el algoritmo continuaría reduciendo el intervalo hasta que el límite izquierdo superara al derecho.

Es importante ordenar los datos antes de aplicar búsqueda binaria, porque el algoritmo necesita que los elementos estén organizados según el mismo criterio de comparación utilizado durante la búsqueda.

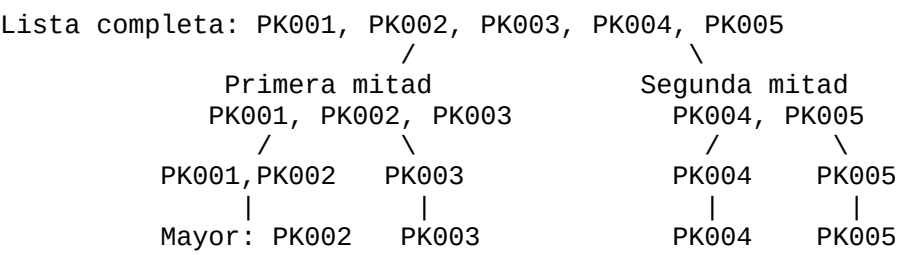
C.2.4. Divide y vencerás: paquete de mayor peso

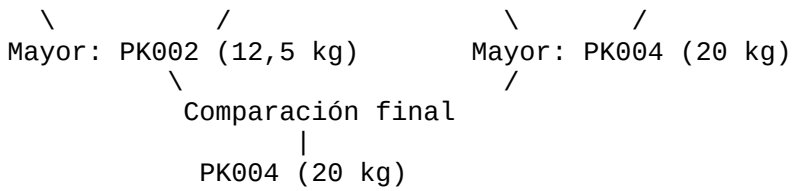
La clase *PaqueteMayorPeso.java* divide la lista en dos mitades, encuentra recursivamente el paquete más pesado de cada mitad y compara los resultados para conservar el de mayor peso.

Para ilustrarlo, se utiliza la lista de la prueba *testEncuentraPaqueteMasPesado():*

Código	Peso
PK001	5,0 kg
PK002	12,5 kg
PK003	8,0 kg
PK004	20,0 kg
PK005	3,5 kg

El proceso de división y combinación puede representarse así:





El resultado esperado es **PK004, con un peso de 20 kg**. El caso base ocurre cuando el intervalo contiene un único paquete. En cada combinación se compara el peso de los dos candidatos y se devuelve el más pesado. Si ambos tienen el mismo peso, la implementación conserva el que aparece primero en la lista original.

C.3. Análisis de complejidad temporal y espacial

En la siguiente tabla, (n) representa la cantidad de paquetes procesados.

Operación	Complejidad temporal	Complejidad espacial adicional	Justificación
Buscar un paquete en el `HashMap` por código	(O(1)) esperado	(O(1)) por consulta	Se accede al valor mediante su clave; el costo es esperado y puede aumentar si hay colisiones.
Sumar pesos por destino mediante recursividad	(O(n))	(O(n))	Se revisa cada paquete y se utiliza una llamada recursiva por elemento.
Contar paquetes por prioridad mediante recursividad	(O(n))	(O(n))	Cada paquete se compara una vez con la prioridad mínima.
Búsqueda binaria sobre una lista ya ordenada	(O(\log n))	(O(1)) en una implementación iterativa	En cada paso se descarta aproximadamente la mitad del intervalo.
Método `buscarPorCodigo()` de la implementación actual	(O(n\log n))	(O(n))	Primero copia y ordena la lista; luego realiza la búsqueda binaria.
Encontrar el paquete de mayor peso mediante divide y vencerás	(O(n))	(O(\log n))	Se resuelven dos mitades recursivamente y se combinan sus resultados.
Agregar un elemento al final de la lista enlazada propia	(O(1))	(O(1)) por inserción	La lista mantiene referencias al primer y al último nodo.
Consultar un elemento de la lista enlazada por índice	(O(n))	(O(1)) adicional	Se recorren los nodos hasta alcanzar la posición solicitada.

La búsqueda binaria tiene una complejidad de (O(\log n)) cuando recibe datos ya ordenados. Sin embargo, la implementación actual de `buscarPorCodigo()` ordena una copia en cada consulta. Por eso, su complejidad total es (O(n\log n)), y no (O(\log n)).

Si el sistema necesitara consultar repetidamente paquetes por código, *el HashMap<String, Paquete> de CentroDistribucion.java* sería una alternativa más eficiente para esas consultas. La búsqueda binaria se conserva para demostrar el algoritmo solicitado en el parcial.

La suma y el conteo recursivos consumen (O(n)) espacio adicional en la pila de llamadas porque la recursión avanza elemento por elemento. En cambio, el algoritmo de divide y vencerás para encontrar el paquete de mayor peso divide el intervalo aproximadamente por la mitad en cada nivel, por lo que la profundidad de la recursión es (O(\log n)).

C.4. Verificación de pruebas y resultados

La solución incluye pruebas automatizadas para verificar el comportamiento de las estructuras de datos, los algoritmos implementados y las operaciones del centro de distribución. Entre los principales casos evaluados se encuentran:

- Suma recursiva de pesos para un destino determinado.

- Destinos sin paquetes registrados, cuyo peso total debe ser cero.
- Comparación de destinos sin distinguir entre mayúsculas y minúsculas.
- Conteo recursivo de paquetes que cumplen una prioridad mínima.
- Manejo de listas vacías y argumentos nulos, según las validaciones de cada método.
- Búsqueda de paquetes mediante códigos existentes y no existentes.
- Identificación del paquete de mayor peso, incluyendo listas vacías, listas con un solo elemento y empates de peso.
- Funcionamiento de la lista enlazada y del historial de entregas.
- Validaciones durante el registro de paquetes y repartidores.
- Asignación y entrega de paquetes, así como el funcionamiento de las colas FIFO y de prioridad.
- Consistencia entre los estados de los paquetes y las estructuras de datos del sistema.

En la ejecución de *mvn test* correspondiente a la versión revisada del proyecto se obtuvieron los siguientes resultados:

Indicador	Resultado
Pruebas ejecutadas	79
Fallos	0
Errores	0
Pruebas omitidas	0
Resultado final	*BUILD SUCCESS*

Estos resultados evidencian que las 79 pruebas automatizadas finalizaron satisfactoriamente, sin fallos, errores ni pruebas omitidas. Para conservar la validez de esta evidencia, cualquier modificación posterior del código debe someterse nuevamente a pruebas, registrando los resultados de la versión definitiva que se entrega.

Por otra parte, la ejecución de la aplicación principal, mediante la clase *App.java*, permitió comprobar el funcionamiento integrado del sistema con un conjunto de 30 paquetes y 6 repartidores. Los principales resultados obtenidos fueron los siguientes:

Operación verificada	Resultado
Paquetes registrados	30
Repartidores registrados	6
Peso total de los paquetes destinados a Armenia	47,0 kg
Paquetes con prioridad mayor o igual a 4	12
Código del paquete de mayor peso	PK019
Peso del paquete de mayor peso	20,0 kg
Entregas registradas en la demostración	3
Consistencia de las estructuras de datos	Sí

Los resultados anteriores corresponden al conjunto de datos utilizado en la ejecución principal y son independientes de los resultados obtenidos en las pruebas de escritorio, donde se emplean conjuntos más pequeños para facilitar el seguimiento manual de los algoritmos.

En conclusión, las pruebas de escritorio permiten comprender paso a paso el funcionamiento de los algoritmos, mientras que las pruebas automatizadas verifican su comportamiento ante diferentes escenarios y condiciones. A su vez, la ejecución principal demuestra la integración de las estructuras de datos y las operaciones de gestión de paquetes. En conjunto, estas verificaciones proporcionan evidencia del funcionamiento de los

componentes evaluados y permiten relacionar la implementación práctica con el análisis algorítmico y los objetivos académicos de la asignatura.

.

.

D.1. Bitácora de uso de inteligencia artificial

Durante el desarrollo y la revisión del proyecto QuindíoExpress se utilizó una herramienta de inteligencia artificial como apoyo para comprender las estructuras de datos, analizar la complejidad de las operaciones, revisar decisiones de implementación y verificar resultados. Las sugerencias obtenidas se contrastaron con el código fuente, las pruebas unitarias y la ejecución del programa.

Consulta o actividad	Dificultad que motivó la consulta	Uso de la respuesta	Verificación realizada
Comparación entre ArrayList y HashMap	Determinar qué estructura resulta más adecuada para consultar paquetes mediante su código.	Apoyo para analizar las diferencias de complejidad temporal y consumo de memoria.	Se contrastó la explicación con las estructuras utilizadas en el código.
Análisis de la memoria utilizada por las estructuras de datos.	Comprender el costo adicional de utilizar un mapa en lugar de una lista.	Explicó que un *HashMap* necesita administrar asociaciones entre claves y valores, mientras que un *ArrayList* mantiene referencias en un arreglo dinámico.	Se conservó la conclusión de que el mapa suele utilizar más memoria auxiliar, pero mejora el tiempo esperado de búsqueda por código.
Selección de una estructura para mantener el orden de llegada de los paquetes.	Garantizar que el primer paquete registrado sea el primero en retirarse mediante el mecanismo FIFO.	Explicó el comportamiento de las colas FIFO y su diferencia frente a una cola de prioridad.	Se verificó el uso de Queue<Paquete> respaldada por ArrayDeque<Paquete> y se contrastó con las operaciones de retiro del programa.
Análisis de la cola de prioridad y sus criterios de comparación.	Establecer qué paquete debe atenderse primero cuando existen diferentes prioridades y tiempos estimados.	Ayudó a interpretar el orden definido mediante un comparador personalizado.	Se revisó *ComparadoresPaquete.PARA_DESPACHO:* prioridad descendente, tiempo estimado ascendente y código ascendente para desempatar.
Diferencias entre HashSet y TreeSet.	Evitar municipios repetidos y, al mismo tiempo, disponer de una colección ordenada para las consultas.	Explicó que HashSet evita duplicados sin garantizar el orden de recorrido, mientras que TreeSet mantiene un orden definido por su comparador.	Se contrastó con las estructuras utilizadas para controlar las claves de municipios y obtener los municipios ordenados.
Comprensión de la lista simplemente enlazada genérica.	Entender cómo almacenar el historial de entregas sin sustituir la implementación propia por una colección estándar.	Explicó el papel de los nodos, las referencias al primer y último nodo y la relación entre la lista y el iterador.	Se revisó ListaEnlazada<T>, que implementa ListaSimple<T> e Iterable<T>, y se verificó que las operaciones y

			pruebas correspondieran a su implementación.
Análisis del algoritmo recursivo para sumar pesos por destino.	Comprender cómo dividir el problema en casos recursivos y acumular únicamente los pesos de los paquetes que coinciden con el municipio solicitado.	Ayudó a identificar el caso base, el avance del índice y la condición de coincidencia del destino.	Se contrastó con sumarPesoPorDestino y se comprobó el resultado de 47.0 kg para Armenia en la ejecución de demostración.
Análisis del conteo recursivo por prioridad mínima.	Determinar cómo contar los paquetes cuya prioridad es igual o superior a un valor establecido.	Explicó la condición de conteo y el retorno recursivo hasta alcanzar el final de la lista.	Se revisó contarPorPrioridadMinima y se contrastó el resultado de 12 paquetes con prioridad igual o superior a 4 en la demostración.
Revisión de la búsqueda binaria de paquetes por código.	Comprender la necesidad de ordenar los datos antes de aplicar búsqueda binaria y evitar modificar innecesariamente la lista original.	Explicó el funcionamiento de la búsqueda binaria y la importancia de trabajar con elementos ordenados.	Se verificó que BusquedaPaquetes crea una copia, la ordena por código y realiza la búsqueda sobre esa copia. Se tuvo en cuenta que el método completo incluye el costo de ordenar.
Análisis del algoritmo para encontrar el paquete de mayor peso.	Comprender la estrategia de divide y vencerás y la comparación de los resultados obtenidos en cada mitad.	Ayudó a interpretar la división recursiva de la lista y la selección del paquete de mayor peso.	Se revisó PaqueteMayorPeso y se contrastó la ejecución con el resultado PK019, de 20.0 kg.
Revisión de la asignación de paquetes y el control de estados.	Comprender las condiciones necesarias para asignar un paquete a un repartidor y registrar una entrega sin inconsistencias.	Ayudó a organizar conceptualmente las transiciones entre estados y las restricciones de asignación.	Se contrastó la explicación con CentroDistribucion, especialmente las condiciones de retiro, disponibilidad del repartidor, coincidencia de zona y registro de entregas.
Interpretación de las pruebas unitarias.	Determinar si el funcionamiento de los componentes estaba respaldado por pruebas y no solamente por una demostración de consola.	Ayudó a interpretar los resultados de las pruebas y a relacionarlos con los comportamientos evaluados.	Se ejecutó mvn test y se obtuvo el resultado registrado de 79 pruebas, 0 fallos, 0 errores y 0 omitidas.
Verificación de resultados numéricos de la demostración.	Confirmar que los valores obtenidos mediante los algoritmos coincidieran con los datos registrados.	La herramienta proporcionó inicialmente un total incorrecto para el peso de los paquetes destinados a Armenia.	Se revisaron individualmente los seis paquetes del destino, se recalculó la suma y se comprobó la salida del programa. El valor correcto fue 47.0 kg.
Preparación de la documentación técnica del proyecto.	Explicar las decisiones de diseño y las complejidades sin atribuir	Ayudó a organizar las justificaciones de las estructuras de datos, las	Se contrastó el contenido con el código revisado y con las ejecuciones

	al programa funciones que no implementa.	complejidades y los resultados de las pruebas.	registradas, corrigiendo las afirmaciones que no coincidían con la implementación.

Conclusión de la bitácora

El uso de inteligencia artificial facilitó la comprensión de conceptos relacionados con las estructuras de datos, la recursividad, la búsqueda, la complejidad algorítmica y las pruebas de software. Sin embargo, el proceso también evidenció que las respuestas generadas pueden contener errores y que una explicación aparentemente convincente no garantiza un resultado correcto.

Por esta razón, las decisiones finales se fundamentaron en la revisión del código fuente, la ejecución de la aplicación y los resultados de las pruebas unitarias. La herramienta se utilizó como apoyo para comprender y documentar el proyecto, mientras que la validación de los resultados se realizó mediante evidencia verificable.

D.2. Análisis de una respuesta cuestionable de inteligencia artificial

Campo	Descripción
Respuesta original	La herramienta de inteligencia artificial indicó inicialmente que el peso total de los paquetes destinados a Armenia era de 63 kg.
Problema detectado	El resultado numérico era incorrecto y no coincidía con la suma de los pesos de los paquetes registrados para ese destino.
Análisis	Al revisar los paquetes correspondientes a Armenia, se identificaron los códigos PK001, PK006, PK011, PK016, PK021 y PK026, cuyos pesos son 2, 7, 12, 17, 2 y 7 kg, respectivamente. La suma correcta es 2 + 7 + 12 + 17 + 2 + 7 = 47 kg. La ejecución del programa también mostró un resultado de 47.0 kg.
Solución final	Se aceptó el resultado de 47.0 kg, respaldado por el cálculo individual de los paquetes y por la salida real del programa.

Análisis de la respuesta cuestionable

¿Qué propuso la herramienta?

La herramienta proporcionó un total de 63 kg para los paquetes destinados a Armenia.

¿Por qué la solución no es apropiada?

Porque el resultado no coincidía con la suma de los pesos de los paquetes que tenían ese destino. La respuesta se había presentado sin una comprobación suficiente del cálculo.

Complejidad, memoria, comportamiento y estructura utilizada

El error correspondía a la exactitud del resultado numérico, no a una selección inadecuada de estructura de datos ni a una complejidad algorítmica incorrecta. El procedimiento implementado para sumar los pesos utiliza recursividad y recorre la lista de paquetes para acumular los valores que coinciden con el destino indicado.

¿Qué decidieron implementar?

Se conservó el método recursivo *sumarPesoPorDestino* y se verificó el resultado con los datos del programa. La ejecución de QuindíoExpress confirmó un total de 47.0 kg para Armenia. Por tanto, se rechazó la cifra inicial de la IA y se aceptó el resultado comprobado.

Conclusión

El desarrollo del proyecto QuindíoExpress permitió integrar y aplicar los fundamentos de las estructuras de datos y el análisis de algoritmos en la solución de un problema de gestión y distribución de paquetes. A través de la implementación de listas, mapas, conjuntos, colas FIFO, colas de prioridad y una lista enlazada propia, se evidenció que la selección de una estructura de datos debe responder a las necesidades específicas de cada operación y no únicamente a su facilidad de implementación. De esta manera, el proyecto relaciona los conceptos teóricos estudiados con una aplicación funcional que contempla el registro, la búsqueda, la asignación y la entrega de paquetes.

Desde la perspectiva algorítmica, la implementación de métodos recursivos, búsqueda binaria y divide y vencerás permitió analizar diferentes formas de resolver problemas computacionales. La construcción de pruebas de escritorio facilitó la comprensión de la pila de llamadas, los casos base y la combinación de resultados, mientras que el análisis de complejidad temporal y espacial permitió identificar las ventajas y limitaciones de cada procedimiento. En particular, se reconoció la importancia de distinguir la complejidad de un algoritmo aislado de la complejidad total de una operación, como ocurre con la búsqueda binaria cuando previamente es necesario copiar y ordenar los datos.

La verificación mediante pruebas automatizadas constituyó otro componente fundamental del trabajo, puesto que permitió evaluar el comportamiento de las estructuras y los algoritmos ante diferentes condiciones de entrada. En la ejecución reportada, las 79 pruebas finalizaron sin fallos ni errores, y la demostración principal produjo resultados verificables, como un peso total de 47,0 kg para los paquetes destinados a Armenia y la identificación de PK019 como el paquete de mayor peso, con 20,0 kg. Estos resultados respaldan el funcionamiento de los escenarios evaluados, sin implicar que las pruebas garanticen la ausencia absoluta de defectos en cualquier situación posible.

En términos académicos, el proyecto permitió comprender que la calidad de una solución informática depende de la articulación entre el diseño de las estructuras de datos, la elección de los algoritmos, el análisis de su eficiencia y la verificación sistemática de sus resultados. Asimismo, la documentación técnica y las pruebas de escritorio fortalecieron la capacidad de explicar y justificar las decisiones de implementación, aspectos esenciales en el desarrollo de software.

En conclusión, QuindíoExpress representa la aplicación práctica de los conocimientos adquiridos en la asignatura de Estructura de Datos y demuestra la importancia de abordar los problemas computacionales mediante soluciones organizadas, verificables y técnicamente fundamentadas. El proyecto establece una base para futuras ampliaciones y, principalmente, evidencia que comprender el funcionamiento interno y el costo de las operaciones es tan importante como obtener el resultado esperado.